

OpenStack Fundamentals

Part 2 of 2

*Advanced Operations
& HPC Integration*

ADSAI – HCP and Cloud Course

University of Trieste

Academic Year 2025/2026

Topics: Security Groups • Network Deep Dive • Storage Patterns • Security Best Practices
• Troubleshooting • HPC Integration Preview

Security Groups: Stateful Virtual Firewalls

Applied at the Neutron port level: analogous to AWS Security Groups

Default State	
INBOUND <i>DENY ALL</i>	No inbound traffic reaches the VM unless explicitly permitted by a rule
OUTBOUND <i>ALLOW ALL</i>	All outbound traffic is permitted by default. Override with explicit deny.
STATEFUL <i>RETURN TRAFFIC</i>	Established connections automatically allow return packets - no explicit egress rule needed for inbound sessions
vs. AWS Security Groups Concept is identical. Key difference: AWS SGs attach to ENIs; OpenStack SGs attach to Neutron ports. Both are stateful. Both default-deny inbound.	

Rule Anatomy	
Field	Options / Example
Direction	ingress / egress
Protocol	tcp / udp / icmp / any
Port range	22 (SSH), 80:443 (HTTP/S), 10000:10999 (MPI)
Remote IP	0.0.0.0/0 or specific CIDR (e.g. 10.0.0.0/8)
Remote group	Name of another security group (for intra-cluster)
Ethertype	IPv4 / IPv6
<pre>openstack security group create cluster-sg openstack security group rule create \ --protocol tcp --dst-port 22 cluster-sg openstack security group rule list cluster-sg</pre>	

Security Group Design: HPC Cluster Pattern

Principle of least privilege - purpose-specific groups, intra-group references

head-node-sg

- SSH TCP 22 from institution CIDR (e.g. 193.204.0.0/16)
- ICMP from institution CIDR (ping for diagnostics)
- All traffic from worker-sg (job submission, MPI)
- HTTP/S 80:443 from 0.0.0.0/0 (monitoring dashboard)

worker-sg

- All traffic from head-node-sg (SLURM control, SSH tunnels)
- All traffic from worker-sg (MPI all-to-all, RDMA)
- NFS/NFS-lock TCP 2049,111 from storage-sg
- NO direct SSH from internet - use head node as jump host

storage-sg

- NFS TCP/UDP 2049,111 from worker-sg
- NFS TCP/UDP 2049,111 from head-node-sg
- SSH TCP 22 from head-node-sg (admin only)
- NO public internet access

Key: use `--remote-group worker-sg` instead of broad CIDR ranges for intra-cluster rules - automatically adapts as workers are added/removed

Neutron: Complete Network Topology Deep Dive

Provider networks → Virtual router → Tenant subnets → Floating IPs

EXTERNAL / PROVIDER NETWORK (CINECA-managed, routable - e.g. 10.0.0.0/8 campus range)

VIRTUAL ROUTER (external gateway +
SNAT/DNAT + Floating IP pool)

TENANT NETWORK (VXLAN-encapsulated) - Subnet: 192.168.100.0/24 • DHCP enabled • DNS: 8.8.8.8

Head Node

IP: 192.168.100.10

SG: head-node-sg

FIP: Floating IP

Cinder vol.

Worker-1

IP: 192.168.100.11

SG: worker-sg

FIP: -

Cinder vol.

Worker-2

IP: 192.168.100.12

SG: worker-sg

FIP: -

Cinder vol.

Storage VM

IP: 192.168.100.20

SG: storage-sg

FIP: -

Cinder vol.

VXLAN isolation: tenant networks are Layer 2 segments tunnelled over the physical IP fabric. VNI (24-bit) gives 16M isolated networks - same technology as AWS VPC.

Neutron: Complete Network Stack - CLI Reference

From zero to SSH-accessible VM: 8 commands

```
# === 1. Create private network ===
openstack network create my-research-network --description 'Private net for HPC cluster'

# === 2. Create subnet with DHCP and DNS ===
openstack subnet create --network my-research-network \
  --subnet-range 192.168.100.0/24 --gateway 192.168.100.1 \
  --dns-nameserver 8.8.8.8 --dns-nameserver 8.8.4.4 my-subnet

# === 3. Create router + attach external gateway ===
openstack router create my-router
openstack router set --external-gateway external-network my-router
openstack router add subnet my-router my-subnet

# === 4. Allocate floating IP and assign to head node ===
FIP=$(openstack floating ip create external-network -f value -c floating_ip_address)
PORT=$(openstack port list --server head-node -f value -c id | head -1)
openstack floating ip set --port $PORT $FIP
echo "Head node reachable at: ssh ubuntu@$FIP"
```

openstack network list | openstack router show my-router | openstack floating ip list - verify at each step

Neutron vs. AWS VPC: Detailed Comparison

Conceptual parity - operational differences at every level

Feature	AWS VPC	OpenStack Neutron
Network creation	<code>aws ec2 create-vpc --cidr-block</code>	<code>openstack network create</code>
Subnets	VPC subnets, explicit AZ assignment	Subnets attached to networks, no AZ constraint
Internet connectivity	Explicit Internet Gateway (IGW) object	Router with external gateway + SNAT
Public IP	Elastic IP (EIP) - persistent, allocated	Floating IP - same concept, same model
Routing	Explicit per-subnet route tables	Handled by virtual router (L3 agent)
Security	Security Groups (per-ENI) + NACLs (per-subnet)	Security Groups (per-port) only - no NACL equivalent
DNS resolution	Route 53 / VPC DNS (169.254.169.253)	Neutron DHCP agent or external DNS
Transparency	Abstracted, limited topology control	Full visibility and control over virtual topology
Network isolation	VPC boundaries, cross-VPC peering	VXLAN VNI isolation, shared L3 router possible
HPC optimization	Placement groups (cluster/spread)	Host aggregates + availability zones

Missing in Neutron: no direct equivalent of AWS Network Load Balancer (NLB), AWS PrivateLink, or VPC Flow Logs in base installation - require additional OpenStack services (Octavia, etc.)

Storage Architecture: The Full Picture

Three storage tiers - choosing the right one for each workload

Nova Ephemeral (Root Disk)	Cinder (Block Volume)	Swift (Object Storage)
• Lives on compute node local disk • Destroyed when VM is terminated • Best performance (local NVMe on ADA) • Use for: OS, temporary scratch, staging	• Independent of VM lifecycle • Can be detached and reattached • Ceph-backed on ADA (replicated 3×) • Use for: databases, checkpoints, model weights	• HTTP REST API (S3-compatible endpoints) • Unlimited scale, pay-per-byte-stored • No mount - access via openstack object • Use for: datasets, results, images, archives
Best for: Short-lived batch jobs with no persistent state	Best for: Any data that must survive VM failure or deletion	Best for: Shared access, large datasets, workflow inputs/outputs

Swift: Object Storage for Research Data

HTTP REST API, S3-compatible - staging datasets across VM workflows

Container	Top-level namespace (≈ S3 bucket). Globally unique within the project. No nested containers.
Object	Arbitrary binary blob with metadata. Key = container/object-name. Max single object: 5 GB (larger via SLO).
SLO/DLO	Static/Dynamic Large Objects: multi-part upload for files >5 GB. Essential for genomics datasets.
ACLs	Container-level access control: .rlistings (list), .r:* (public read). Cross-project sharing possible.

Swift CLI Workflow

```
# Create a container
openstack container create \
  mhpc-genomics-data

# Upload dataset
openstack object create \
  mhpc-genomics-data \
  /data/genome_hg38.fa

# List objects
openstack object list mhpc-genomics-data

# Download on worker VM
openstack object save \
  mhpc-genomics-data \
  genome_hg38.fa

# Make container public (readonly)
openstack container set \
  --property 'X-Container-Read=.r:*' \
  mhpc-genomics-data
```

Cinder Advanced: Snapshots and Volume-Backed Instances

Data durability patterns - essential for production research workflows

Snapshot Workflow - Checkpoint Before Risk

Running Simulation
(Cinder vol. attached)

Checkpoint:
openstack volume snapshot create

Risky Operation
(update, migration)

If failure:
openstack volume create --snapshot

Volume-Backed Instances - Persistent OS

- OS resides on a Cinder volume, NOT ephemeral disk
- VM can be deleted and recreated without data loss
- Volume can be resized independently of flavor
- Enable with: `--boot-from-volume` or `--volume <vol-id>`
- Essential for long-running research nodes (weeks/months)
- Snapshot the boot volume before OS upgrades

```
# Create bootable volume from image
openstack volume create \
  --image 'Ubuntu 22.04' \
  --size 50 --bootable \
  my-boot-vol
```

```
# Boot from volume (OS persists!)
openstack server create \
  --flavor m1.large \
  --volume my-boot-vol \
  --key-name mhpc-key \
  persistent-research-node
```

Cinder vs. AWS EBS: Feature Comparison

Block storage parity - backend-dependent capabilities on private cloud

Feature	AWS EBS	OpenStack Cinder (ADA)
Volume types	gp3, io2, st1, sc1 (explicit spec)	Depends on backend: LVM, Ceph, NFS
ADA backend	-	Ceph (distributed, 3× replication, CRUSH)
Snapshots	Yes, stored in S3 (cross-region copy)	Yes, stored in Swift or Ceph object
Multi-attach	io1/io2 only (with constraints)	Backend-dependent (Ceph supports it)
Encryption	KMS-managed (transparent)	Barbican-managed (if deployed)
Max size	64 TB	Backend-limited (Ceph: PB scale)
IOPS control	Explicit (gp3: 3,000–16,000 IOPS)	Volume type (QoS policy by admin)
Volume resize	Extend only (no shrink)	Extend only
Detach/reattach	Only within same AZ	Any compute node in same availability zone
Cost	Per-GB-month + per-IOPS	Quota-based (no per-use billing on ADA)

Ceph backend on ADA: replicated 3× across nodes - more durable than single-AZ EBS. Zero-cost snapshots (CoW). Consistent throughput without IOPS provisioning.

Security Best Practices in OpenStack

You own the security of everything inside your project - CINECA does not manage your VMs

Credentials

- Never commit openrc.sh or credentials to Git
- Use Application Credentials (scoped, revocable, time-limited)
- Rotate credentials regularly; revoke on project end
- Store in environment variables or secrets manager, never in code

SSH Access

- Use Ed25519 or RSA ≥ 4096 -bit key pairs - no password auth
- Restrict SSH access to institution CIDR, not 0.0.0.0/0
- Workers behind head node (jump host) - no direct public SSH
- Use ssh-agent forwarding or ProxyJump, not copying private keys

Network

- Principle of least privilege: purpose-specific security groups
- No default security group on worker/storage VMs
- Use --remote-group for intra-cluster rules, not broad CIDR
- Review openstack security group rule list regularly

Data

- Encrypt sensitive data at rest (LUKS, application-level)
- Snapshot Cinder volumes before any destructive operation
- Set container ACLs on Swift - avoid accidental public exposure
- Delete unused floating IPs (they are a limited resource on ADA)

Troubleshooting: Systematic Debugging Approach

8 common failure modes - always start with openstack server show

Symptom	Likely Cause	Diagnosis Command
VM stuck in BUILD state	Image download slow, compute node busy	openstack server show <vm>; openstack console log show <vm>
VM in ERROR state	Quota exceeded, flavor unavailable, network fault	openstack server show <vm> → fault.message field
Cannot SSH (Connection refused)	Security group missing port 22 rule	openstack security group rule list <sg>
Cannot SSH (Timeout)	No floating IP / wrong IP / routing issue	openstack floating ip list; ping <fip>
Volume attachment fails	Volume in 'in-use' by another server	openstack volume show <vol> → attachments
Network unreachable inside VM	Router not attached to subnet	openstack router show; openstack port list
Auth token expired (403)	Token TTL 1h exceeded	source openrc.sh (re-authenticate); check OS_AUTH_URL
Quota exceeded (403/Over Limit)	Project vCPU/RAM/IP quota hit	openstack quota show; openstack limits show

Debug order: server show → console log → security group rules → network topology → quota → platform status page (adacloud.hpc.cineca.it)

Quota Management on CINECA ADA Cloud

Shared infrastructure - quota limits protect all users; plan before provisioning

Quota Categories

Resource	Typical Limit
Instances (VMs)	10–20 per project
vCPUs	40–80 total
RAM	100–200 GB
Floating IPs	2–5 per project
Security Groups	10 per project
Cinder volumes	10 volumes, 500 GB total
Swift storage	Project-dependent (TB range)
Networks	5 per project
Routers	2 per project

Quota CLI & Planning

```
# Check your project quota
openstack quota show

# Check current usage
openstack limits show --absolute

# Check hypervisor capacity (admin)
openstack hypervisor stats show

# List all your running VMs
openstack server list --all-projects
```

Planning for the Exercise

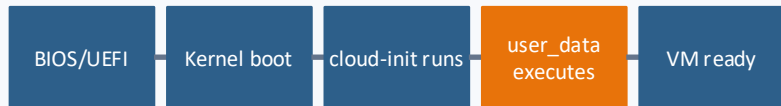
- Always destroy resources after each session
- Never leave floating IPs unassigned - return to pool
- Use m1.small for testing, scale up for benchmarks
- openstack server delete + openstack volume delete before logging off

Cloud-Init: Automating VM Initialization

user_data scripts run once at first boot - foundation of Infrastructure as Code

What is cloud-init?

- Standard mechanism: injected at VM launch via metadata API
- Runs once at first boot as root
- Used by all major cloud images (Ubuntu, CentOS, Rocky)
- Metadata endpoint: `http://169.254.169.254/latest/meta-data/`
- Analogous to AWS EC2 user data - identical concept



Boot timeline (seconds)

user_data Example: BLAST Worker Setup

```
#!/bin/bash
set -e # Exit on error

# Install BLAST tools
dnf install -y ncbi-blast+ \
  openmpi openmpi-devel

# Mount Cinder data volume
mkdir -p /data /results
mount /dev/vdb /data

# Download reference database from Swift
openstack object save \
  mhpc-genomics-data nr_db.tar.gz
tar xzf nr_db.tar.gz -C /data/
```

```
openstack server create --user-data ./setup.sh --image 'CentOS-8' --flavor m1.xlarge blast-worker-1
```

Debug user_data: `openstack console log show blast-worker-1` or `cat /var/log/cloud-init-output.log` on the VM

End-to-End Workflow: MPI Cluster on CINECA ADA

Combining all concepts - from CLI to running parallel job

1. Auth & Quota

```
source openrc.sh
openstack quota show
openstack flavor list
```

2. Network Stack

```
network create
subnet create
router create + attach
```

3. Security Groups

```
head-node-sg (SSH+22)
worker-sg (intra-cluster)
storage-sg (NFS only)
```

4. Storage

```
volume create --size 200
(shared NFS on head node)
Swift for input datasets
```

5. Launch VMs

```
head node (m1.xlarge)
+ N workers (m1.large)
with user_data scripts
```

6. Configure

```
SSH → head node
Mount volumes
Generate MPI hostfile
```

7. Run MPI Job

```
mpirun -np 8 \
  -hostfile hosts \
  ./mpi_app
```

8. Cleanup!

```
server delete (all)
volume delete
floating ip delete
```

⚠ ALWAYS CLEAN UP: Delete all VMs, volumes, and floating IPs after your session. Leaving resources running wastes shared quota and may incur costs on AWS exercises.

HPC–Cloud Integration: Three Patterns

OpenStack as a private cloud - how it connects to the broader HPC ecosystem

1. Interactive Analysis Via Cloud VM

Researcher deploys Jupyter/RStudio on OpenStack VM. Connects to data on Swift object store. Cinder volume for persistent work. Head node handles SSH port-forwarding.

✓ Pros:

- Flexible resources, no queue
- Direct browser access
- Persistent workspace

✗ Cons:

- Manual provisioning
- Quota consumed while idle

2. Pre/Post-Processing HPC + Cloud

Heavy MPI/SLURM computation on CINECA HPC cluster. Pre/post-processing (format conversion, visualization, statistics) on OpenStack cloud VM. Data exchange via Lustre-mount or Swift.

✓ Pros:

- Best tool for each stage
- HPC for tightly coupled
- Cloud for elastic post-proc

✗ Cons:

- Data transfer overhead
- Two platforms to manage

3. Cloud Bursting: Parametric Sweeps

Baseline runs on CINECA HPC. When queue is full or extra capacity needed: burst to OpenStack (or AWS if allowed). BLAST/MD/parameter searches - embarrassingly parallel workloads.

✓ Pros:

- Eliminates queue wait
- Massive embarrassing-parallel
- Cost-effective for burst

✗ Cons:

- Data staging complexity
- Requires automation (Heat/Terraform)

Preview: Infrastructure as Code - The Next Step

Manual CLI is necessary for learning - but production requires automation

The manual CLI problem: You just typed 25 commands to build a 4-VM cluster. What if you need it again tomorrow? Or 10 identical environments for different experiments? Or need to prove to a collaborator exactly how it was built?

Heat (OpenStack-native) (≈ CloudFormation)

- HOT (Heat Orchestration Template) - YAML-based
- Describes resources and their dependencies as a DAG
- `openstack stack create -t cluster.yaml`
- Native: deep integration with all OpenStack services
- `openstack stack delete` - removes ALL resources atomically

Terraform (Multi-cloud) (≈ CDK/Terraform for AWS too)

- HCL (HashiCorp Configuration Language) - declarative
- OpenStack provider: [github.com/terraform-provider-openstack](https://github.com/terraform-providers/terraform-provider-openstack)
- `terraform plan` → `terraform apply` → `terraform destroy`
- State file tracks what's deployed - reproducible by design
- Same tool works for OpenStack AND AWS (Lecture 2!)

Lecture 7 (OpenStack Advanced): Heat HOT templates • Complete Terraform module for HPC cluster • Hybrid cloud bursting patterns

Glance: Image Management for Scientific VMs

Custom images accelerate deployment - pre-install HPC tools, drivers, libraries

Public Images

Provided by CINECA: CentOS 8 Stream, Ubuntu 22.04, Rocky Linux. Minimal OS, cloud-init installed. Starting point for customization.

Private Images

Snapshots of your configured VMs. Visible only to your project. Fastest way to replicate configured environments.

Community Images

Shared across projects on ADA. Scientific images shared by research groups (BLAST+OpenMPI, GROMACS, custom HPC stack).

Snapshot workflow: (1) launch base image → (2) install HPC tools → (3) `openstack server image create` → (4) use snapshot as base for all workers

Glance CLI Workflow

```
# List available images
openstack image list --public
openstack image list --shared

# Show image details
openstack image show 'Ubuntu 22.04'

# Create snapshot of running VM
openstack server image create \
  --name 'blast-worker-v1' \
  blast-analysis-vm

# Upload custom image (qcow2)
openstack image create \
  --disk-format qcow2 \
  --container-format bare \
  --file custom-hpc-image.qcow2 \
  mhpc-hpc-base-v2

# Delete old image (free quota)
openstack image delete mhpc-hpc-base-v1
```

Lecture 1: Complete Takeaways

Private Cloud WHY

Data sovereignty (GDPR), TCO at scale, hardware control, HPC integration. OpenStack = dominant OSS platform in research.

Modular Architecture

12+ independent services, all REST-API driven. Keystone is the trust anchor. RabbitMQ for async coordination.

Nova+Neutron+Cinder

Launch VMs, build network stacks, attach persistent volumes. Mirrors AWS EC2+VPC+EBS but you operate it.

Security Groups

Stateful, per-port virtual firewalls. Default-deny inbound. Design purpose-specific groups. Use --remote-group for intra-cluster.

Storage Tiers

Ephemeral (fast, temporary) → Cinder block (persistent, per-VM) → Swift object (shared, scalable). Match tier to workload.

IaC Preview

Manual CLI for learning; Heat + Terraform for production. Reproducibility and automation start from day one.

Lecture 2 → Heat Orchestration Templates • Terraform for OpenStack • Swift Advanced • Hybrid HPC-Cloud Burst Patterns • Troubleshooting Workshop